

Alard Charitable Trust's

Alard College of Engineering & Management

AIML Department

LAB MANUAL

Name of Faculty: Prof. Shradha Pawar

Subject: Operating System Lab 1

Subject Code: PCC-205-AIM

Class: Third Year

Academic Year: 2026-27

Semester: 5th Semester

Index

Sr. No.	Title
1	Linux Environment and File Management
2	Shell Scripting and System Automation
3	Process Creation, Scheduling, and Performance Analysis
4	Multithreading and Resource Utilization
5	Inter-Process Communication and Shared Resource Management
6	Synchronization Mechanisms for Concurrent Systems
7	Deadlock Detection, Avoidance, and Recovery
8	Memory Management and Page Replacement Strategies
9	File System and Disk Scheduling Analysis
10	Containerization and Resource Monitoring for AI/ML Workloads

Assignment No. 1

Title : Linux Environment and File Management

Objectives:

To understand and work with the Linux operating system environment and command-line interface by exploring the Linux file system hierarchy, performing file and directory operations, managing file permissions and ownership, monitoring processes and system resources, and developing familiarity with the Linux terminal for operating system programming.

Problem Statement:

Perform file and directory operations in Linux using command-line utilities. Demonstrate file permissions, ownership management, process monitoring, and basic system administration commands.

Software and Hardware Requirements:

Software: Windows 10/11 with WSL2 **OR** Ubuntu Virtual Machine,Bash Terminal
Hardware: Computer with minimum 2GB RAM, 20GB HDD

Theory:

1. Introduction to Linux

Linux is an open-source operating system based on the Unix operating system principles.
It provides:

- Process management
- Memory management
- File management
- Device management
- Security
- Networking
- User management

Linux can be operated using:

1. **Graphical User Interface (GUI)**
2. **Command-Line Interface (CLI)**

In this experiment, the **Linux CLI** is used.
The command line allows users to directly communicate with the operating system by entering commands in the terminal.

2.Linux File System

Linux follows a hierarchical file system.
The top-level directory is called the **root directory**, represented by: " / "
Important directories include:

Directory	Purpose	Directory	Purpose
/	Root directory	/root	Home directory of root user
/home	User home directories	/var	Variable data and logs
/bin	Essential executable commands	/usr	User programs and utilities
/etc	System configuration files	/dev	Device files
/tmp	Temporary files	/proc	Process and kernel information

3. Linux Command Structure
A typical Linux command has the following structure:

```
command [options] [arguments]
```

Example:
ls -l /home
Here:

- ls → command
- -l → option
- /home → argument

A. Navigation and Directory Commands

These commands are used to navigate through and manage directories.

Command	Purpose	Example
pwd	Displays the current working directory	pwd
ls	Lists files and directories	ls -l
cd	Changes the current directory	cd Documents
mkdir	Creates a new directory	mkdir Data
rmdir	Removes an empty directory	rmdir Data

B. File Management Commands

These commands are used to create and manipulate files.

Command	Purpose	Example
touch	Creates an empty file	touch notes.txt
cp	Copies files/directories	cp a.txt Backup/
mv	Moves or renames files	mv a.txt b.txt
rm	Deletes files	rm a.txt
cat	Displays file contents	cat notes.txt

C. File Viewing Commands

These commands are used to view the contents of files.

Command	Purpose	Example
cat	Displays complete file contents	cat file.txt
head	Displays beginning of a file	head file.txt
tail	Displays end of a file	tail file.txt

D. File and Text Searching Commands

Command	Purpose	Example
find	Searches for files/directories	find . -name "*.txt"
grep	Searches for text inside files	grep "Linux" notes.txt

E. File Permission and Ownership Commands

Linux provides permissions to control access to files and directories.

Command	Purpose	Example
chmod	Changes file permissions	chmod 644 file.txt
chown	Changes file owner/group	sudo chown user file.txt
ls -l	Displays permissions and ownership	ls -l

Linux Permissions

Symbol	Meaning	Numeric Value
r	Read	4
w	Write	2
x	Execute	1

F. Process Management Commands

These commands are used to view and manage running processes.

Command	Purpose	Example
ps	Displays running processes	ps
top	Displays processes in real time	top
kill	Terminates a process	kill 1234

G. System Information Commands

Command	Purpose	Example
uname	Displays system information	uname -a
whoami	Displays current username	whoami
date	Displays current date and time	date

H. Disk and Storage Management Commands

Command	Purpose	Example
df	Displays available disk space	df -h
du	Displays space used by files/directories	du -sh OSLab

I. Memory Monitoring Commands

Command	Purpose	Example
free	Displays RAM and swap usage	free -h

Students Should Perform below Tasks and attach the print of all executed commands.

- Create a directory named `OSLab`.
- Create three subdirectories: `Data`, `Programs`, and `Output`.
- Create three files inside `Data`: `student.txt`, `marks.txt`, and `notes.txt`.
- Add appropriate text content to each file.
- Display the contents of all three files.
- Copy `student.txt` to the `Output` directory.
- Rename `marks.txt` to `result.txt`.
- Move `result.txt` to the `Output` directory.
- Search for `student.txt` using the `find` command.
- Search for a specific word in `student.txt` using the `grep` command.
- Change the permission of `student.txt` to `644` using `chmod`.
- Display and verify the file permissions using `ls -l`.
- Display currently running processes using `ps`.
- Display detailed process information using `ps -ef`.
- Display system information using `uname -a`.
- Display the current logged-in user using `whoami`.
- Display the current date and time using `date`.
- Display memory usage using `free -h`.
- Display available disk space using `df -h`.
- Display the disk usage of the `OSLab` directory using `du -sh`.
- Display the complete directory structure of `OSLab`.

Conclusion

Thus, various Linux file and directory operations were successfully performed using command-line utilities. File permissions, ownership, process monitoring, disk usage, memory information and basic system administration commands were also demonstrated successfully.

Assignment No. 2

Title :Shell Scripting and System Automation

Objectives:

To understand and implement Linux shell scripting using variables, command-line arguments, decision-making statements, loops, functions, and file/directory operations to automate common operating system tasks.

Problem Statement:

Develop shell scripts using decision-making statements, loops, functions, and file handling commands to automate common operating system tasks.

Software and Hardware Requirements:

Software: Windows 10/11 with WSL2 **OR** Ubuntu Virtual Machine,Bash Terminal
Hardware: Computer with minimum 2GB RAM, 20GB HDD

Theory:

1. Introduction to Shell Scripting

A **shell script** is a file containing a sequence of Linux commands that are executed together. It is used to automate repetitive tasks such as file management, system monitoring, backups, and administration.

Bash is a commonly used Linux shell.

A shell script generally starts with:

#!/bin/bash

2. Creating and Executing a Shell Script

Create a script:
touch script.sh
Edit it using VS Code:
code script.sh
Example:
#!/bin/bashecho "Operating System Lab"
Give execute permission:
chmod +x script.sh
Execute the script:
./script.sh
or
bash script.sh

3. Shell Variables

Category: Variable Management
Variables store values.
name="Student"echo $name
Note: No spaces are used around =.

4. User Input

Category: Input
The read command accepts input from the user.
read -p "Enter your name: " nameecho "Hello $name"

5. Command-Line Arguments

Category: Input / Script Parameters
Arguments are values passed while executing a script.
./script.sh student.txt

Variable	Meaning
$0	Script name
$1	First argument
$2	Second argument
$#	Number of arguments
$@	All arguments

6. Decision-Making Statements

Category: Decision Making

if

```
if [ condition ]
then
   Commands
Fi

Example:
if [ -f "$file" ]
then
   echo "File exists"
fi
```

if-else

```
if [ -d "$dir" ]
then
   echo "Directory exists"
   else
   echo "Directory does not exist"
fi
```

if-elif-else
Used to check multiple conditions.

```
if [ condition1 ]
then
   Commands
   elif [ condition2 ]
   then
   Commands
   else
   Commands
   fi
```

Category: Multi-way Decision Making

Used for menu-based choices.

```
case $choice in
   1) echo "Create File" ;;
   2) echo "Delete File" ;;
   3) echo "Display File" ;;
   *) echo "Invalid Choice" ;;esac
```

6. File Test Operators

Category: File Testing

Operator	Purpose
-f	Regular file exists
-d	Directory exists
-e	Path exists
-r	File is readable
-w	File is writable
-x	File is executable
-s	File is non-empty

Example:

```
if [ -f "$file" ]then
   echo "File exists"
fi
```

7. Loops

Category: Iteration / Looping

for Loop

Used to repeat operations for a list of items.

```
for file in *.txt
do
   echo "$file"
done
```

while Loop

Executes while the condition is true.

```
count=1
while [ $count -le 5 ]
do
   echo $count
   count=$((count+1))
done
```

until Loop

Executes until the condition becomes true.

```
count=1
until [ $count -gt 5 ]
do
   echo $count
   count=$((count+1))
done
```

8. Shell Functions

Category: Modular Programming

A function is a reusable block of commands.

```
display_message()
{
   echo "Operating System Lab"
}

display_message
```

Functions reduce code repetition and improve program organization.

9. Arithmetic Operations

Category: Arithmetic
Arithmetic expressions are evaluated using $(( )).
a=10b=5sum=$((a+b))echo "Sum = $sum"
Common operators:

Operator	Operation
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus

Students Should Perform below Tasks and attach the print of all executed commands/Programs.
Develop a Bash script that:

1. Accepts a directory path from the user.
2. Checks whether the directory exists.
3. Displays files and directories.
4. Counts files and directories.
5. Displays directory size.
6. Provides a menu for:
    1. Create file
    2. Display file
    3. Copy file
    4. Rename file
    5. Delete file
    6. Search file
    7. Display directory information
    8. Exit
    9. Uses **functions,** `if-else`**,** `case`**, loops and file-handling commands**

Conclusion:

Thus, a Bash shell script was successfully developed using decision-making statements, loops, functions and file-handling commands to automate common operating system tasks.

Assignment No. 3

Title : Process Creation, Scheduling, and Performance Analysis

Objectives:

To understand process creation, identifiers, and states, implement process management and CPU scheduling algorithms such as **FCFS, SJF, and Round Robin**, calculate scheduling performance metrics including waiting time, turnaround time, and response time, and compare the performance of different scheduling algorithms.

Problem Statement:

Develop programs to create and manage processes using system calls. Implement CPU scheduling algorithms (FCFS, SJF, Round Robin) and analyze their impact on process execution, memory utilization, and overall system performance.

Software and Hardware Requirements:

Software: Windows 10/11 with WSL2 **OR** Ubuntu Virtual Machine,Bash Terminal
Hardware: Computer with minimum 2GB RAM, 20GB HDD

Theory:

A **process** is a program currently in execution. Each process has a unique **Process ID (PID)**.

Process States

State	Meaning
New	Process is being created
Ready	Waiting for CPU
Running	Using CPU
Waiting	Waiting for an event/I/O
Terminated	Execution completed

Process System Calls

System Call	Category	Purpose
fork()	Process Creation	Creates a child process
getpid()	Process Identification	Returns current PID
getppid()	Process Identification	Returns parent PID
wait()	Synchronization	Waits for child process
exit()	Termination	Terminates process
exec()	Program Execution	Executes another program

CPU Scheduling

CPU scheduling decides **which ready process gets the CPU next**.

Important Parameters

Parameter	Formula
Completion Time (CT)	Time at which process finishes
Turnaround Time (TAT)	CT - AT
Waiting Time (WT)	TAT - BT
Response Time (RT)	First CPU start time - AT

Where:

- AT = Arrival Time
- BT = Burst Time

1. FCFS Scheduling
Full Form: First Come First Serve
Category: Non-Preemptive
The process that arrives first is executed first.
Algorithm

1. Arrange processes according to arrival time.
2. Execute the first process completely.
3. Execute the next process.
4. Calculate CT, TAT and WT.

Example:
P1 → P2 → P3

2. SJF Scheduling
Full Form: Shortest Job First
Category: Non-Preemptive
The process with the **shortest burst time** is selected first.
Algorithm

1. Find available processes.
2. Select the process with the smallest burst time.
3. Execute it completely.
4. Repeat until all processes finish.
5. Calculate CT, TAT and WT.

3. Round Robin Scheduling
Category: Preemptive
Each process gets a fixed amount of CPU time called **Time Quantum**.
If the process is not completed, it goes to the end of the queue.
Algorithm

1. Enter processes and burst times.
2. Enter time quantum.
3. Execute a process for the given quantum.
4. If incomplete, place it at the end of the queue.
5. Repeat until all processes finish.
6. Calculate CT, TAT and WT.

Students Should Perform below Tasks and attach the print of all executed commands/Programs(with output).

Write and execute a C program to:

1. Create a child process using fork() and display the **PID and PPID**.
2. Use wait() for **parent-child synchronization**.
3. Define three processes (P1, P2, P3) with burst times **5, 3, and 2**.

4. Implement **FCFS scheduling** and calculate **Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT)**.
5. Display the **FCFS Gantt chart**.
6. Implement **SJF scheduling** and display its **CT, TAT, WT and Gantt chart**.
7. Implement **Round Robin scheduling** with **Time Quantum = 2** and display its execution sequence and Gantt chart.
8. Compare **FCFS, SJF, and Round Robin** based on their process execution and scheduling behavior.
9. Compile and execute the program and attach the **output screenshot**.

Conclusion:

Thus, processes were successfully created using Linux system calls, and FCFS, SJF and Round Robin CPU scheduling algorithms were implemented and analyzed using scheduling parameters.

Assignment No. 4

Title : Multithreading and Resource Utilization

Objectives:

To understand and implement multithreading using POSIX threads for executing multiple tasks concurrently.
To measure execution time and compare the performance of the application with different numbers of threads.

Problem Statement:

Create a multithreaded application using POSIX threads to execute concurrent tasks. Measure CPU utilization, memory consumption, and execution time under different thread configurations.

Software and Hardware Requirements:

Software: Ubuntu / WSL / Virtual Machine, VS Code, GCC Compiler,Linux `time` command
Hardware: Computer with minimum 2GB RAM, 20GB HDD

Theory:

A **thread** is the smallest unit of CPU execution within a process.

Multiple threads of the same process can execute concurrently and share resources such as:

- Code
- Data
- Files
- Memory

Process vs Thread:

Process	Thread
Independent execution unit	Execution unit within a process
Has separate memory space	Shares process memory
More resource-intensive	Less resource-intensive
Process creation is slower	Thread creation is faster

POSIX Threads:

POSIX Threads (Pthreads) is a standard API for creating and managing threads in Linux.

Include the header:

#include <pthread.h>

Important Functions

Function	Category	Purpose
pthread_create()	Thread Creation	Creates a thread
pthread_join()	Thread Synchronization	Waits for a thread
pthread_exit()	Thread Termination	Terminates a thread
pthread_self()	Thread Identification	Returns thread ID

Algorithm

1. Start.
2. Create a large array.
3. Enter the number of threads.
4. Divide the array into equal parts.
5. Create the required threads.
6. Each thread calculates the sum of its assigned part.
7. Wait for all threads using pthread_join().
8. Add the partial sums.
9. Display the final result.
10. Measure execution time and resource utilization.
11. Repeat for different thread counts.
12. Stop.

Students Should Perform below Tasks and attach the print of all executed commands/Programs(with output).

Develop a multithreaded C program to perform a computational task such as **calculating the sum of a large array**. The program should:

1. Create an array containing a large number of elements.
2. Divide the array into parts.
3. Assign each part to a separate thread.
4. Calculate the partial sum concurrently.
5. Combine the partial results.
6. Execute the program using different numbers of threads.
7. Measure execution time and CPU/memory usage.
8. Compare the results.

Conclusion:

Thus, a multithreaded application was successfully developed using POSIX threads, and its execution time, CPU utilization and memory consumption were measured for different thread configurations

Assignment No. 5

Title : Inter-Process Communication and Shared Resource Management

Objectives:

To understand and implement Inter-Process Communication (IPC) using **pipes, shared memory, and message queues** & compare their communication efficiency, resource sharing, and memory utilization.

Problem Statement:

Implement Inter-Process Communication using pipes, shared memory, and message queues.
Analyze communication efficiency, resource sharing, and memory utilization

Software and Hardware Requirements:

Software: Ubuntu / WSL / Virtual Machine, VS Code, GCC Compiler, Linux commands
Hardware: Computer with minimum 2GB RAM, 20GB HDD

Theory:

Inter-Process Communication (IPC) is a mechanism that allows two or more processes to communicate and exchange data with each other.

Processes normally have separate memory spaces. IPC mechanisms provide a way for processes to share or transfer information.

Common IPC mechanisms include:

- Pipes
- Shared Memory
- Message Queues

Pipe:

A **pipe** is a simple IPC mechanism used to transfer data between processes.

It provides a communication channel through which one process can write data and another process can read it.

Shared Memory:

Shared memory allows two or more processes to access the same memory area.

It is useful when processes need to exchange a large amount of data efficiently.

Message Queue:

A **message queue** allows processes to communicate by sending and receiving messages through a queue maintained by the operating system.

Comparison of IPC Mechanisms:

Pipe	Shared Memory	Message Queue
Transfers data through a communication channel	Processes share a common memory area	Transfers data in the form of messages
Simple to implement	Faster for large data transfer	Suitable for structured messages
Data is copied between processes	Data can be accessed directly from shared memory	Messages are managed by the operating system
Used for simple communication	Useful for efficient resource sharing	Useful for message-based communication

Important Functions / System Calls:

Function / System Call	Category	Purpose
pipe()	Pipe	Creates a pipe for communication
read()	Pipe	Reads data from the pipe
write()	Pipe	Writes data to the pipe
shmget()	Shared Memory	Creates or accesses shared memory
shmat()	Shared Memory	Attaches shared memory to a process
shmdt()	Shared Memory	Detaches shared memory
msgget()	Message Queue	Creates or accesses a message queue
msgsnd()	Message Queue	Sends a message
msgrcv()	Message Queue	Receives a message

Algorithm:

1. Start.
2. Create two processes using fork().
3. Implement communication between processes using a **pipe**.
4. Send data from one process and receive it in another process.
5. Implement communication using **shared memory**.
6. Write data into shared memory from one process and read it from another process.
7. Implement communication using a **message queue**.
8. Send and receive a message between the processes.
9. Observe the communication and resource sharing for each method.
10. Compare the three IPC mechanisms based on efficiency and memory utilization.
11. Stop.

Students Should Perform below Tasks and attach the print of all executed commands/Programs(with output).

Develop C programs to demonstrate **Inter-Process Communication using pipes, shared memory, and message queues**.

The programs should:

1. Create two processes using fork().
2. Implement communication between the processes using a **pipe**.
3. Implement data sharing between the processes using **shared memory**.
4. Implement message exchange between the processes using a **message queue**.
5. Execute all three programs successfully.

6. Observe and record the communication results.
7. Compare the three IPC mechanisms based on:
    1. Communication efficiency
    2. Resource sharing
    3. Memory utilization
8. Prepare a short comparison table based on the observations.

Conclusion:

Thus, Inter-Process Communication was successfully implemented using pipes, shared memory, and message queues. The three IPC mechanisms were studied and compared based on communication efficiency, resource sharing, and memory utilization.

Assignment No. 6

Title : Synchronization Mechanisms for Concurrent Systems

Objectives:

To understand and implement synchronization using semaphores and mutex locks for managing concurrent processes or threads. To study the effect of synchronization on resource allocation and system performance.

Problem Statement:

Develop programs using semaphores and mutex locks to solve synchronization problems such as Producer–Consumer and Readers–Writers. Evaluate the effect of synchronization on resource allocation and system performance

Software and Hardware Requirements:

Software: Windows 10/11 with WSL2 **OR** Ubuntu Virtual Machine,Bash Terminal
Hardware: Computer with minimum 2GB RAM, 20GB HDD

Theory:

Synchronization is a mechanism used to control the execution of multiple processes or threads when they access shared resources.

When multiple processes or threads access the same resource at the same time, incorrect results may occur. This situation is called a **race condition**.

Synchronization mechanisms ensure that shared resources are accessed safely and in a controlled manner.

Common synchronization mechanisms include:

- Mutex Locks
- Semaphores

Mutex Lock:

A **mutex (Mutual Exclusion) lock** allows only one thread to access a critical section at a time.

It prevents multiple threads from modifying shared data simultaneously.

Semaphore:

A **semaphore** is a synchronization variable used to control access to shared resources.

It can be used to indicate whether a resource is available or occupied.

Producer–Consumer Problem:

The **Producer–Consumer problem** involves a producer that generates data and a consumer that uses the data.

Synchronization is required to ensure that:

- The producer does not add data when the buffer is full.
- The consumer does not remove data when the buffer is empty.

- Both do not access the shared buffer incorrectly.

Readers–Writers Problem:

The **Readers–Writers problem** involves multiple processes accessing a shared resource.

- Multiple readers can read the resource simultaneously.
- Only one writer can modify the resource at a time.
- A writer should not access the resource while another reader or writer is using it.

Comparison of Synchronization Mechanisms:

Mutex Lock	**Semaphore**
Provides mutual exclusion	Controls access to resources
Usually allows one thread at a time	Can allow one or more threads depending on its value
Used to protect critical sections	Used for synchronization and resource management
Simple for protecting shared data	Useful for problems such as Producer–Consumer

Important Functions / System Calls:

Function	**Category**	**Purpose**
pthread_create()	Thread Creation	Creates a new thread
pthread_join()	Thread Synchronization	Waits for a thread to finish
pthread_mutex_init()	Mutex	Initializes a mutex
pthread_mutex_lock()	Mutex	Locks the critical section
pthread_mutex_unlock()	Mutex	Unlocks the critical section
pthread_mutex_destroy()	Mutex	Destroys the mutex
sem_init()	Semaphore	Initializes a semaphore
sem_wait()	Semaphore	Decreases semaphore value / waits
sem_post()	Semaphore	Increases semaphore value / signals
sem_destroy()	Semaphore	Destroys the semaphore

Algorithm:

1. Start.
2. Create multiple threads using pthread_create().
3. Create and initialize a mutex lock.
4. Use the mutex to protect the shared resource.
5. Implement the **Producer–Consumer** problem using semaphores.
6. Initialize the required semaphores.
7. Allow the producer to add data and the consumer to remove data safely.
8. Implement the **Readers–Writers** problem using synchronization mechanisms.
9. Control access to the shared resource by readers and writers.
10. Observe the effect of synchronization on resource access and execution.
11. Compare the performance and resource utilization.
12. Stop.

Students Should Perform below Tasks and attach the print of all executed commands/Programs(with output).

Develop **C programs to demonstrate synchronization using mutex locks and semaphores**.

The programs should:

1. Create multiple threads using pthread_create().
2. Implement **mutual exclusion using a mutex lock**.
3. Implement the **Producer–Consumer problem using semaphores**.
4. Implement the **Readers–Writers problem using suitable synchronization mechanisms**.
5. Execute all programs successfully.
6. Observe and record the synchronization results.
7. Compare the programs based on:
    1. Resource sharing
    2. Synchronization mechanism used
    3. Execution time
    4. CPU utilization
    5. Memory utilization
8. Prepare a short comparison table based on the observations.

Observation Table:

Program	**Synchronization Used**	**Execution Time**	**CPU Usage**	**Memory Usage**
Mutex	Mutex Lock			
Producer–Consumer	Semaphore			
Readers–Writers	Mutex/Semaphore			

Conclusion:

Thus, synchronization mechanisms were successfully implemented using **mutex locks and semaphores**. The **Producer–Consumer and Readers–Writers problems** were studied, and the effect of synchronization on resource allocation, shared resource access, and system performance was analyzed.

Assignment No. 7

Title :
Deadlock Detection, Avoidance, and Recovery

Objectives:
To understand the concept of deadlock in operating systems. To implement **Banker's Algorithm** for deadlock avoidance and a **simple deadlock detection technique**. To analyze resource allocation patterns and determine whether the system is in a safe or deadlock state.

Problem Statement:
Implement Banker's Algorithm and deadlock detection techniques. Analyze resource allocation patterns and evaluate system performance under different deadlock scenarios.

Software and Hardware Requirements:

Software: Ubuntu / WSL / Virtual Machine, VS Code, GCC Compiler, Linux commands

Hardware: Computer with minimum 2GB RAM, 20GB HDD

Theory:

Deadlock

Deadlock is a situation in which two or more processes wait indefinitely for resources held by each other.

Example:

P1 → Waiting for R2
R2 → Held by P2

P2 → Waiting for R1
R1 → Held by P1

Deadlock Avoidance

Deadlock avoidance checks resource allocation before granting resources to ensure that the system remains in a **safe state**.

Banker's Algorithm is used for deadlock avoidance.

Banker's Algorithm

Banker's Algorithm checks whether the available resources are sufficient to satisfy the remaining requirements of processes.

The main data used are:

- **Available:** Resources currently available.
- **Maximum:** Maximum resources required by each process.
- **Allocation:** Resources currently allocated to each process.
- **Need:** Remaining resources required.

The Need matrix is calculated as:

Need = Maximum − Allocation

If all processes can complete in some order, the system is in a **Safe State**.

Deadlock Detection

Deadlock detection checks whether the system has already entered a deadlock state.

If some processes cannot complete because the required resources are not available, those processes may be involved in a deadlock.

Deadlock Recovery

After detecting a deadlock, the operating system can recover by terminating selected processes or releasing resources so that other processes can continue.

Comparison of Deadlock Handling Techniques:

Technique	Working Principle	Main Feature
Deadlock Detection	Checks whether deadlock has occurred	Identifies deadlocked processes
Deadlock Avoidance	Checks resource allocation before granting resources	Prevents unsafe allocation
Banker's Algorithm	Checks for a safe sequence	Avoids deadlock
Deadlock Recovery	Takes action after deadlock is detected	Releases resources / terminates processes

Conclusion:

The concepts of **deadlock detection and avoidance** were studied and implemented. **Banker's Algorithm** was used to identify the safe state and safe sequence, while the **deadlock detection algorithm** was used to identify deadlocked processes. The experiment helped understand how proper **resource allocation and management** can prevent and detect deadlock conditions in an operating system.

Assignment No. 8

Title : Memory Management and Page Replacement Strategies

Objectives:

To understand and simulate **paging, segmentation, and virtual memory management** and implement **FIFO, LRU, and Optimal page replacement algorithms** to compare their effectiveness using performance metrics.

Problem Statement:

Simulate paging, segmentation, and virtual memory management. Implement **FIFO, LRU, and Optimal page replacement algorithms** and compare their performance based on **page faults, page hits, hit ratio, and fault ratio**.

Software and Hardware Requirements:

Software: Ubuntu / WSL / Virtual Machine, VS Code, GCC Compiler, Linux commands

Hardware: Computer with minimum 2GB RAM, 20GB HDD

Theory:

Memory Management is an operating system function that manages the allocation and deallocation of memory to processes.

Memory management techniques help the operating system use memory efficiently and provide memory to processes when required.

Common memory management techniques include:

- Paging
- Segmentation
- Virtual Memory
- Page Replacement

Paging:

Paging is a memory management technique in which logical memory is divided into fixed-size **pages** and physical memory is divided into fixed-size **frames**.

A logical address consists of:

- Page Number
- Offset

Segmentation:

Segmentation divides a program into logical sections called **segments**, such as code, data, and stack.

A logical address consists of:

- Segment Number
- Offset

Virtual Memory:

Virtual memory allows a process to execute even when the complete process is not present in physical memory.

When a required page is not available in memory, a **page fault** occurs.

Page Replacement:

When all available memory frames are occupied and a new page needs to be loaded, the operating system replaces an existing page using a **page replacement algorithm**.

Common page replacement algorithms include:

- FIFO
- LRU
- Optimal

Comparison of Page Replacement Algorithms:

FIFO	LRU	Optimal
Replaces the oldest page	Replaces the least recently used page	Replaces the page that will be used farthest in the future
Simple to implement	Uses recent page usage	Requires future reference information
May produce more page faults	Generally gives better results than FIFO for many reference strings	Used as an ideal/reference algorithm
Easy to understand	Requires tracking page usage	Difficult to implement in a real system

Important Concepts / Terms:

Term	Category	Purpose
Page	Paging	Fixed-size block of logical memory
Frame	Paging	Fixed-size block of physical memory
Page Table	Paging	Maps pages to frames
Page Fault	Virtual Memory	Occurs when required page is not in memory
Page Hit	Virtual Memory	Occurs when required page is already in memory
Segment	Segmentation	Logical part of a program
FIFO	Page Replacement	Replaces the oldest page
LRU	Page Replacement	Replaces the least recently used page
Optimal	Page Replacement	Replaces the page used farthest in the future
Hit Ratio	Performance Metric	Measures successful page accesses
Fault Ratio	Performance Metric	Measures page faults

Algorithm:

1. Start.
2. Simulate paging using pages, frames, and a page table.
3. Simulate segmentation using segment numbers, base addresses, and limits.
4. Understand the occurrence of page faults in virtual memory.
5. Read the page reference string and number of frames.

6. Implement the **FIFO page replacement algorithm**.
7. Implement the **LRU page replacement algorithm**.
8. Implement the **Optimal page replacement algorithm**.
9. Calculate page hits and page faults for each algorithm.
10. Calculate hit ratio and fault ratio.
11. Compare the performance of all three algorithms.
12. Stop.

Students Should Perform below Tasks and attach the print of all executed commands/Programs(with output).

Develop **C programs to demonstrate memory management and page replacement techniques**.

The programs should:

1. Implement the **FIFO page replacement algorithm**.
2. Implement the **LRU page replacement algorithm**.
3. Implement the **Optimal page replacement algorithm**.
4. Execute all programs successfully.
5. Record the number of **page hits and page faults**.
6. Calculate and compare:
    1. Page Faults
    2. Page Hits
    3. Hit Ratio
    4. Fault Ratio
7. Prepare a short comparison table based on the observations.

Performance Metrics:

Hit Ratio:

Hit Ratio = Page Hits / Total Page References

Fault Ratio:

Fault Ratio = Page Faults / Total Page References

Observation Table:

Technique / Algorithm	Page Hits	Page Faults	Hit Ratio	Fault Ratio
Paging				
Segmentation				
FIFO				
LRU				
Optimal				

Conclusion:

Thus, **paging, segmentation, and virtual memory management** were successfully simulated. The **FIFO, LRU, and Optimal page replacement algorithms** were implemented and compared based on page hits, page faults, hit ratio, and fault ratio.

Assignment No. 9

Title : File System and Disk Scheduling Analysis

Objectives:

To understand basic file allocation methods and disk scheduling techniques. To implement **FCFS, SSTF, SCAN, and C-SCAN** disk scheduling algorithms using simple programs and compare their performance based on **head movement, access time, and throughput**.

Problem Statement:

Implement file allocation and disk scheduling algorithms (FCFS, SSTF, SCAN, CSCAN).
Analyze storage utilization, access time, and system throughput.

Software and Hardware Requirements:

Software: Ubuntu / WSL / Virtual Machine,VS Code,GCC Compiler,Linux commands
Hardware: Computer with minimum 2GB RAM, 20GB HDD

Theory:

File Allocation

File allocation defines how files are stored on disk blocks. It helps the operating system manage and access files efficiently.

Common file allocation methods are:

1. **Contiguous Allocation:** File blocks are stored in consecutive disk blocks.
2. **Linked Allocation:** Each block contains a link to the next block.
3. **Indexed Allocation:** An index block stores the addresses of all file blocks.

For this experiment, students can implement a **simple contiguous file allocation simulation**.

Disk Scheduling

Disk scheduling determines the order in which disk I/O requests are served. The main objective is to reduce disk head movement and access time.

FCFS – First Come First Serve

Requests are served in the same order in which they arrive.

Example:
Requests: 98 183 37 122
Order: 98 → 183 → 37 → 122

It is very simple but may result in more head movement.

SSTF – Shortest Seek Time First

The request closest to the current disk head position is served first.

It generally reduces head movement compared with FCFS.

SCAN

The disk head moves in one direction and services requests along the way. After reaching the end, it changes direction.

It is also called the **Elevator Algorithm**.

C-SCAN – Circular SCAN

The disk head moves in one direction and services requests. After reaching the end, it returns to the beginning and continues in the same direction.

Comparison of Disk Scheduling Algorithms:

Algorithm	Working Principle	Main Feature
FCFS	Serves requests in arrival order	Very simple
SSTF	Serves nearest request first	Reduces head movement
SCAN	Moves in one direction and then reverses	More uniform service
C-SCAN	Moves in one direction and returns to start	More uniform waiting time

Important Functions / Concepts:

Category	Concept / Function	Purpose
File Allocation	Contiguous Allocation	Stores file blocks continuously
File Allocation	Disk Block	Represents a storage block
Disk Scheduling	FCFS	Services requests in given order
Disk Scheduling	SSTF	Services nearest request
Disk Scheduling	SCAN	Services requests while moving in one direction
Disk Scheduling	C-SCAN	Services requests in circular order
Performance	Head Movement	Measures total disk head movement
Performance	Access Time	Time required to access a request
Performance	Throughput	Number of requests completed in a given time

Algorithm:

Part A – Simple File Allocation

1. Start.
2. Create an array representing disk blocks.
3. Enter the starting block and number of blocks required for a file.
4. Check whether the required blocks are free.
5. Allocate consecutive blocks to the file.
6. Display the allocated blocks.
7. Stop.

Part B – Disk Scheduling

1. Start.
2. Enter the disk request queue.
3. Enter the initial disk head position.
4. Implement **FCFS** and calculate total head movement.
5. Implement **SSTF** by selecting the nearest request.
6. Implement **SCAN** by servicing requests in one direction and then reversing.
7. Implement **C-SCAN** by servicing requests in one direction and then moving back to the beginning.
8. Calculate total head movement for each algorithm.
9. Compare the results.
10. Stop.

Students Should Perform Below Tasks and Attach the Print of All Executed Commands/Programs (with Output):

Task 1 – FCFS Disk Scheduling

Write a simple C program to implement **FCFS disk scheduling**.

Use:

Request Queue: 98 183 37 122 14
Initial Head: 53

Calculate:

- Order of servicing
- Total head movement

Task 2 – SSTF Disk Scheduling

Write a simple C program to implement **SSTF disk scheduling**.

For every step, select the request closest to the current head position.

Calculate:

- Order of servicing
- Total head movement

Task 3 – SCAN Disk Scheduling

Write a simple C program to implement **SCAN**.

Use:

Disk Range: 0 – 199
Initial Head: 53
Direction: Right

Calculate:

- Order of servicing
- Total head movement

Keep the program simple by using a **fixed disk range and fixed direction**.

Task 4 – C-SCAN Disk Scheduling

Write a simple C program to implement **C-SCAN**.

Use:

Disk Range: 0 – 199
Initial Head: 53
Direction: Right

Calculate:

- Order of servicing
- Total head movement

The program should move towards the end of the disk and then continue from the beginning.

Task 5 – Performance Comparison

Compare all four disk scheduling algorithms using:

1. Total Head Movement
2. Average Access Time
3. Throughput

Use the following observation table:

Algorithm Total Head Movement Average Access Time Throughput
FCFS
SSTF
SCAN
C-SCAN

Performance Measures:

Total Head Movement:
Total distance travelled by the disk head while servicing all requests.

Average Access Time:

Average Access Time = Total Access Time / Number of Requests

Throughput:

Throughput = Number of Requests Completed / Total Time

Conclusion:

The file allocation and disk scheduling techniques were studied and implemented. **FCFS, SSTF, SCAN, and C-SCAN** were compared using head movement, access time, and throughput. The experiment helps understand how disk scheduling affects storage performance.

Assignment No. 10

Title : Containerization and Resource Monitoring for AI/ML Workloads

Objectives:

To understand the basic concept of containerization and Docker. To deploy a simple **AI/ML application inside a Docker container** and monitor CPU, memory, storage, and process utilization using Docker and Linux commands.

Problem Statement:

Deploy a simple AI/ML application inside a Docker container. Monitor CPU, memory, storage, and process utilization using Linux tools and evaluate the benefits of container based deployment.

Software and Hardware Requirements:

Software: Ubuntu / WSL / Virtual Machine, VS Code, Python 3, Docker, Linux commands
Hardware: Computer with minimum 4GB RAM, 20GB HDD

Theory:

Containerization

Containerization is a method of packaging an application along with its required libraries and dependencies into a single unit called a **container**.

A container provides an isolated environment in which an application can run consistently on different systems.

Docker

Docker is a platform used to create, build, run, and manage containers.

The basic Docker process is:

Dockerfile → Docker Image → Docker Container → Application

Docker Image

A Docker image contains the application, required libraries, dependencies, and instructions needed to run the application.

Docker Container

A container is a running instance of a Docker image.

For example:

Docker Image
↓
Docker Container
↓
AI/ML Application

AI/ML Application

For this experiment, students will use a simple **Python Linear Regression** program.

The program will use sample values to train a basic machine learning model and predict an output.

Resource Monitoring

Resource monitoring means observing the resources used by an application or container.

The main resources considered in this experiment are:

- CPU utilization
- Memory utilization
- Storage utilization
- Running processes

Docker provides commands such as:

docker stats
docker ps
docker top
docker system df

Linux provides commands such as:

top
free
df
ps

Comparison of Container-Based Deployment and Normal Execution:

Method	Working Principle	Main Feature
Normal Execution	Application runs directly on the host system	Simple execution
Docker Container	Application runs inside an isolated container	Isolation and portability
Docker Image	Contains application and required dependencies	Easy deployment
Container Monitoring	Monitors resources used by the container	Resource management

Important Functions / Concepts:

Category	Concept / Function	Purpose
Containerization	Docker	Creates and manages containers
Containerization	Dockerfile	Contains instructions to build an image
Containerization	Docker Image	Packages application and dependencies
Containerization	Docker Container	Runs the application in an isolated environment
AI/ML	Linear Regression	Performs a simple prediction
Monitoring	docker stats	Displays CPU and memory utilization
Monitoring	docker top	Displays processes running in a container

Monitoring	docker system df	Displays Docker storage usage
Monitoring	docker ps	Displays running containers
Linux Monitoring	top	Displays system processes and CPU usage
Linux Monitoring	free -h	Displays memory usage
Linux Monitoring	df -h	Displays disk/storage usage

Algorithm:

Part A – Containerized AI/ML Application

1. Start.
2. Check whether Docker is installed.
3. Create a simple Python AI/ML program.
4. Create a `requirements.txt` file.
5. Create a Dockerfile.
6. Build the Docker image.
7. Run the Docker container.
8. Execute the AI/ML application inside the container.
9. Display the prediction result.
10. Stop the container.
11. Stop.

Part B – Resource Monitoring

1. Start.
2. Run the Docker container.
3. Check the running container using `docker ps`.
4. Monitor CPU and memory using `docker stats`.
5. Display running processes using `docker top`.
6. Check Docker storage usage using `docker system df`.
7. Check system memory using `free -h`.
8. Check disk utilization using `df -h`.
9. Record the observations.
10. Stop.

Benefits of Container-Based Deployment

Study and write the following benefits of containerization:

1. **Portability:** The same container can run on different systems having Docker.
2. **Isolation:** The application runs in an isolated environment.
3. **Easy Deployment:** Application and dependencies are packaged together.
4. **Consistency:** The same environment can be used for development and deployment.
5. **Resource Monitoring:** Docker provides commands to monitor resource utilization.
6. **Easy Management:** Containers can be easily created, started, stopped, and removed.

Conclusion:

The simple AI/ML application was successfully deployed and executed inside a **Docker container**. Docker commands and Linux commands were used to monitor **CPU, memory, storage, and process utilization**. The experiment helped understand the basic concept of **containerization, Docker-based deployment, and resource monitoring for AI/ML workloads**.